

CEFET/RJ
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

GUILHERME BRUCE LAJES GONÇALVES
RAFAEL SANCHES BORGES E SILVA

PROJETO FINAL DE LÓGICA MATEMÁTICA
LINGUAGEM PROLOG

Rio de Janeiro – RJ
08/09/2024

GUILHERME BRUCE LAJES GONÇALVES
RAFAEL SANCHES BORGES E SILVA

PROJETO FINAL DE LÓGICA MATEMÁTICA
LINGUAGEM PROLOG

Relatório apresentado como parte integrante do trabalho em linguagem de programação Prolog contendo uma descrição do código desenvolvido.

Orientação: Diego Nunes Brandão

Rio de Janeiro – RJ

08/09/2024

RESUMO

Por meio deste relatório, procura-se descrever de forma clara e detalhada o funcionamento dos códigos desenvolvidos em linguagem de programação Prolog como respostas às questões propostas para o trabalho, visando a total compreensão acerca das resoluções obtidas. Dessa forma, as questões propostas foram individualmente tratadas ao longo deste documento, cada qual abarcando uma seção contendo o código produzido e outra descrevendo-o, fornecendo explicações para sua interpretação. De resto, as considerações finais deste documento procuram expor de maneira breve as principais dificuldades enfrentadas quando na idealização das respostas e construção do código.

Palavras-chave: descrever, código, Prolog, dificuldades.

SUMÁRIO

1. INTRODUÇÃO.....	6
2. DESENVOLVIMENTO.....	7
2.1. Questão A.....	7
2.1.1. Proposta.....	7
2.1.2. Código.....	7
2.1.3. Descrição.....	7
2.2. Questão B.....	7
2.2.1. Proposta.....	7
2.2.2. Código.....	7
2.2.3. Descrição.....	7
2.3. Questão C.....	8
2.3.1. Proposta.....	8
2.3.2. Código.....	8
2.3.3. Descrição.....	8
2.4. Questão D.....	9
2.4.1. Proposta.....	9
2.4.2. Código.....	9
2.4.3. Descrição.....	9
2.5. Questão E.....	10
2.5.1. Proposta.....	10
2.5.2. Código.....	10
2.5.3. Descrição.....	10
2.6. Questão F.....	10
2.6.1. Proposta.....	10
2.6.2. Código.....	10
2.6.3. Descrição.....	10
2.7. Questão G.....	11
2.7.1. Proposta.....	11
2.7.2. Código.....	11
2.7.3. Descrição.....	11
2.8. Questão H.....	11
2.8.1. Proposta.....	11
2.8.2. Código.....	11
2.8.3. Descrição.....	12

2.9. Questão I	12
2.9.1. Proposta	12
2.9.2. Código	12
2.9.3. Descrição	13
2.10. Questão J	13
2.10.1. Proposta	13
2.10.2. Código	13
2.10.3. Descrição	13
3. CONCLUSÃO	15
4. REFERÊNCIAS	16

1 INTRODUÇÃO

Entende-se por Prolog uma linguagem de programação de escopo declarativo, isto é, um paradigma computacional que descreve o relacionamento entre variáveis em termos de funções ou regras de inferência, não especificando como o algoritmo em si se comportar-se. Afinal, elas aplicam um algoritmo fixo baseado em relações lógicas para produzir um resultado. No caso particular da linguagem Prolog, seus elementos básicos são herdados da lógica de predicados, sendo esses fatos, regras e consultas. (LAGOS, 2003).

Como forma de pôr em prática os conhecimentos adquiridos durante o curso de Lógica Matemática, especialmente relacionados à lógica de predicados, foi proposto a resolução de uma lista de questões em forma de trabalho para avaliar as habilidades dos alunos no assunto referenciado. Nas próximas seções, expor-se-ão as resoluções obtidas.

2 DESENVOLVIMENTO

2.1 Questão A

2.1.1 Proposta

Determinar o número de elementos de uma lista.

2.1.2 Código

nelementos([],0).

nelementos(_|Tail,Num):- nelementos(Tail,Num_temp),Num is Num_temp+1.

2.1.3 Descrição

Para a linha `nelementos([],0).`, define-se como um fato o retorno de 0 para o número de elementos de uma lista vazia, caso base da regra 'nelementos'.

Para a linha `nelementos(_|Tail,Num):- (nelementos(Tail,Num_temp),Num is Num_temp+1).`, inicia-se a regra 'nelementos' de aridade 2, na qual (para qualquer elemento inicial da lista) seus demais elementos (representados pela lista 'Tail') serão contados e armazenados em 'Num'. Com isso, realiza-se uma recursão, enviando a lista sem seu elemento inicial e um contador temporário. Após a recursão atingir uma lista vazia, o valor de 'Num' será incrementado de 0 até a quantidade de recursões realizadas.

2.2 Questão B

2.2.1 Proposta

Determinar o maior valor de uma lista de inteiros.

2.2.2 Código

maior([X],X).

maior([Head|Tail],Maior):- maior(Tail,Maior_temp),((Head>Maior_temp,Maior is Head); (Maior_temp>=Head,Maior is Maior_temp)).

2.2.3 Descrição

Para a linha `maior([X],X).`, Define-se como um fato o retorno de 0 como maior elemento de uma lista vazia, caso base da regra 'maior'.

Para a linha `maior([Head|Tail],Maior):-maior(Tail,Maior_temp), ((Head>Maior_temp,Maior is Head);(Maior_temp>=Head,Maior is Maior_temp)).`, inicia-se a regra 'maior' com aridade 2, na qual 'Head' representa o elemento inicial da lista, 'Tail' uma lista com os elementos restantes e 'Maior' uma variável que armazenará o maior valor. Depois realiza-se uma recursão, enviando a lista sem seu elemento inicial e uma

variável que representará temporariamente o maior valor. Com isso, se o primeiro elemento da lista seja maior que o maior valor até o momento, então o valor deste elemento é atribuído à variável 'Maior', ou senão, se o primeiro elemento da lista for menor que o maior valor até o momento, então o valor de 'Maior_temp' é atribuído à 'Maior'. Essa sequência de análises é repetida para todos os elementos da lista.

2.3 Questão C

2.3.1 Proposta

Determinar o valor médio de uma lista de inteiros.

2.3.2 Código

somaElementos([],0).

somaElementos([Head|Tail],Soma):- somaElementos(Tail,Soma_temp),Soma is Head+Soma_temp.

nelementos([],0).

nelementos([_|Tail],Tamanho):- tamanhoLista(Tail,Tamanho_temp),Tamanho is Tamanho_temp+1.

medio([],0):- !.

medio(Lista,Media):- somaElementos(Lista,Soma),nelementos(Lista,Tamanho), (Tamanho=\=0->Media is Soma/Tamanho;Media is 0).

2.3.3 Descrição

Para a linha somaElementos([],0)., define-se como um fato o retorno de 0 como soma dos elementos de uma lista vazia, caso base da regra 'somaElementos'.

Para a linha somaElementos([Head|Tail],Soma):- somaElementos(Tail,Soma_temp),Soma is Head+Soma_temp., inicia-se a regra 'somaElementos' com aridade 2, na qual 'Head' representa o elemento inicial da lista, 'Tail' uma lista com os elementos restantes e 'Soma' uma variável que armazenará o somatório dos elementos da lista. Depois, realiza uma recursão, enviando a lista sem seu elemento inicial e uma variável que representará temporariamente o somatório. Após a recursão atingir uma lista vazia, a variável 'Soma' funcionará como um acumulador, somando e armazenando uma quantidade de elementos correspondente ao número de recursões executadas.

Para as linhas nelementos([],0). e nelementos([_|Tail],Tamanho):- nelementos(Tail,Tamanho_temp),Tamanho is Tamanho_temp+1., têm-se que o funcionamento desse fato e desta regra já foram explicados no item 2.1.3.

Para a linha `medio([],0):- !.`, define-se como um fato o retorno de 0 para média dos valores de uma lista vazia, além da interrupção (comando de corte '!') do código depois do retorno a fim de evitar uma divisão por 0 no decorrer do código.

Para a linha `medio(Lista,Media):- somaElementos(Lista,Soma),nelementos(Lista,Tamanho),(Tamanho\=0->Media is Soma/Tamanho;Media is 0).`, inicia-se a regra 'medio' com aridade 2, que recebe uma lista de inteiros do usuário e armazenará o valor médio de seus elementos na variável 'Media'. Depois, realiza-se o somatório dos elementos da lista utilizando a regra 'somaElementos' e armazena o resultado na variável 'Soma'. Com isso, o tamanho da lista é aferido utilizando a regra 'tamanhoLista' e armazena-se o resultado na variável 'Tamanho'. Se o tamanho da lista for diferente de zero, então a razão entre as variáveis 'Soma' e 'Tamanho' é armazenada em 'Media'. Senão, a variável 'Media' recebe o valor zero.

2.4 Questão D

2.4.1 Proposta

Inserir um elemento ao final da lista.

2.4.2 Código

`inserirfim(Elemento,[],[Elemento]).`

`inserirfim(Elemento,[Head|Tail_original],[Head|Tail_modificado]):-`

`inserirfim(Elemento,Tail_original,Tail_modificado).`

2.4.3 Descrição

Para a linha `inserirfim(Elemento,[],[Elemento]).`, define-se como um fato o retorno de uma lista unitária à inserção de um elemento em uma lista vazia.

Para a linha `inserirfim(Elemento,[Head|Tail_original],[Head|Tail_modificado]):-` `inserirfim(Elemento,Tail_original,Tail_modificado).`, inicia-se a regra 'inserirfim', com aridade 3, na qual 'Elemento' refere-se ao item a ser inserido no final de uma lista, '[Head|Tail_original]' a lista original à qual deseja-se inserir um apêndice (tendo seu elemento inicial, 'Head', separado dos demais elementos, 'Tail_original'), e '[Head|Tail_modificado]' a lista modificada com 'Elemento' na última posição da lista (tendo seu elemento inicial, 'Head', separado dos demais elementos, 'Tail_modificado').

Assim, realiza-se uma chamada recursiva da regra 'inserirfim', enviando 'Elemento' e as duas listas inseridas sem seu elemento inicial. A recursão persiste até 'Tail_original' corresponder à ultima posição da lista e, em seguida, 'Elemento' é adicionado a esta posição em 'Tail_modificada'.

2.5 Questão E

2.5.1 Proposta

Obter o último elemento da lista.

2.5.2 Código

`ultimo([Elemento],Elemento).`

`ultimo(_|Tail,Elemento):- ultimo(Tail,Elemento),!.`

2.5.3 Descrição

Para a linha `ultimo([Elemento],Elemento).`, define-se como fato o retorno de 'Elemento' para último elemento de uma lista unitária composta por 'Elemento', afinal, é seu único elemento. Outra maneira de definir este fato seria `ultimo(Lista,Elemento) :- Lista=[Elemento]`.

Para a linha `ultimo(_|Tail,Elemento):- ultimo(Tail,Elemento),!.`, inicia-se a regra 'ultimo', de aridade 2, na qual 'Lista' corresponde a um vetor qualquer e 'Elemento' ao item que correspondente ao último elemento da lista. Nela, é executada uma recursão enviando a lista sem seu primeiro elemento, 'Head', até que se atinja uma lista unitária, empregando o fato definido na linha anterior.

2.6 Questão F

2.6.1 Proposta

Verificar se um elemento X é adjacente a um elemento Y.

2.6.2 Código

`adjacente(Elemento1,Elemento2,[Elemento1,Elemento2|_]).`

`adjacente(Elemento1,Elemento2,[Elemento2,Elemento1|_]).`

`adjacente(Elemento1,Elemento2,_|Tail):- adjacente(Elemento1,Elemento2,Tail).`

2.6.3 Descrição

A primeira linha do código define como fato o retorno do valor lógico verdadeiro (*true*) caso 'Elemento1' e 'Elemento2' correspondam aos dois primeiros elementos sequenciais de uma lista, respectivamente, para quaisquer demais elementos dessa lista, '[Elemento1,Elemento2|_]'. Em seguida, na segunda linha, é definido como fato o retorno do valor lógico verdadeiro (*true*) para o caso inverso, isto é, que 'Elemento2' e 'Elemento1' correspondam aos dois primeiros elementos sequenciais de uma lista, respectivamente, para quaisquer demais elementos dessa lista, '[Elemento2,Elemento1|_]'.

Logo após, na terceira linha, inicia-se a regra 'adjacente', de aridade 3, Na qual 'Elemento1' e 'Elemento2' correspondem a dois elementos os quais busca-se saber se são adjacentes em uma listagem, enquanto '[Tail]' a uma lista qualquer, independentemente de seus elementos iniciais, '_', subdividida em outra lista com seus demais elementos 'Tail'. Enfim, a regra realiza uma chamada recursiva, enviando os dois elementos mencionados e a lista sem seus dois elementos iniciais, 'Tail', Dessa forma, a lista será percorrida e uma verificação será realizada para cada par de elementos iniciais da lista de cada recursão, utilizando os fatos definidos anteriormente.

2.7 Questão G

2.7.1 Proposta

Criar uma lista com elementos de um intervalo fechado.

2.7.2 Código

gerar(Inicio,Fim,[]) :- Inicio>Fim.

gerar(Inicio,Fim,[Inicio|Tail]):- Inicio <= Fim, Proximo is Inicio + 1, gerar(Proximo, Fim, Tail).

2.7.3 Descrição

Na primeira linha, define-se como regra, se 'Inicio' for maior que 'Fim', a geração de uma lista vazia, '[]', pois não é possível completar a lista com elementos sequências a partir de 'Inicio' caso este seja maior do que 'Fim', correspondente ao último elemento da lista.

Em seguida, na segunda linha, inicia-se a regra 'gerar', de aridade 3, na qual 'Inicio' corresponde ao elemento inicial da lista que deseja-se gerar, 'Fim' à seu último elemento e '[Inicio|Tail]' à lista que será gerada, separando seu primeiro termo, 'Inicio', de uma lista com os demais elementos 'Tail'. Então, se o valor de 'Inicio' for menor que 'Fim', é atribuído ao próximo elemento, 'Proximo', o valor de 'Inicio' acrescido de 1. Depois, é executada uma chamada recursiva, enviando 'Proximo' como elemento e ser incrementado em seguida, dando continuidade à lista até que 'Proximo' seja maior que 'Fim', finalizando a lista com o elemento correspondente à 'Fim'.

2.8 Questão H

2.8.1 Proposta

Inverter a ordem de uma lista.

2.8.2 Código

inserirfim(Elemento,[],[Elemento]).

`inserirfim(Elemento,[Head|Tail_original],[Head|Tail_modificado]):-`

`inserirfim(Elemento,Tail_original,Tail_modificado).`

`reverter([],[]).`

`reverter([Head|Tail],Lista_invertida):-`

`reverter(Tail,Lista_temp),inserirfim(Head,Lista_temp,Lista_invertida).`

2.8.3 Descrição

Na primeira linha, define-se como fato o retorno de uma lista unitária à inserção de um elemento em uma lista vazia. Afinal, se a lista recebida é vazia, seu único elemento será aquele inserido pelo usuário.

Em seguida, na segunda linha, inicializa-se a regra 'inserirfim', com aridade 3, na qual 'Elemento' refere-se ao item a ser inserido no final de uma lista, '[Head|Tail_original]' à lista original na qual deseja-se inserir um apêndice (tendo seu elemento inicial, 'Head', separado dos demais elementos, 'Tail_original'), e '[Head|Tail_modificado]' à lista modificada com 'Elemento' na sua última posição (tendo seu elemento inicial, 'Head', separado dos demais elementos, 'Tail_modificado').

Finalizada a configuração dessa regra auxiliar, define-se como fato na terceira linha a reversão de uma lista vazia como a si mesma. Afinal, uma lista vazia não possui elementos para serem revertidos.

Logo após, na última linha, inicia-se a regra 'reverter', de aridade 2, na qual '[Head|Tail]' corresponde à lista original (tendo seu elemento inicial, 'Head', separado dos demais, 'Tail'), e 'Lista_invertida' a uma lista de conteúdo análogo à lista original, porém, com a ordenação de seus elementos revertida. Depois, realiza-se uma chamada recursiva, enviando a lista original sem seu primeiro elemento, 'Tail', além de uma lista temporária, 'Lista_temp'. A chamada recursiva persiste até que o último elemento da lista seja atingido e 'Lista_temp' consista em uma lista vazia. Por fim, é realizada uma chamada da regra 'inserirfim', que insere o primeiro elemento da lista original nesta lista temporária já vazia, armazenando esse resultado em 'Lista_invertida'. Esse processo continua para cada recursão realizada até que 'Lista_invertida' seja preenchida pelos elementos da lista original em ordem invertida.

2.9 Questão I

2.9.1 Proposta

Incrementar em uma unidade cada elemento de uma lista de inteiros.

2.9.2 Código

incrementar([],[]).

incrementar([Head|Tail],[Head_incrementado|Tail_incrementado]):-

Head_incrementado is Head+1,incrementar(Tail,Tail_incrementado).

2.9.3 Descrição

Primeiramente, define-se como fato o incremento de uma lista vazia como sendo a própria lista vazia. Afinal, ela não possui elementos para serem acrescidos.

Prosseguindo, inicia-se a regra 'incrementar', de aridade 2, na qual '[Head|Tail]' corresponde à lista original (tendo seu elemento inicial, 'Head', separado dos demais, 'Tail'), enquanto '[Head_incrementado|Tail_Incrementado]' corresponde à lista acrescida em uma unidade em cada elemento (tendo seu elemento inicial, 'Head_modificado', separado dos demais, 'Tail_modificado'). Em seguida, o valor do elemento inicial da lista original acrescido de 1 é atribuído ao elemento inicial da nova lista, 'Head_incrementado'. Por fim, a regra executa uma chamada recursiva, enviando ambas as listas sem seu elemento inicial de forma contínua até que a lista original resume-se a uma lista vazia, atendendo ao fato declarado anteriormente. Nesse processo, cada elemento da lista será acrescido em 1 unidade conforme a recursão avança.

2.10 Questão J

2.10.1 Proposta

Remover todas as ocorrências de um elemento de uma lista.

2.10.2 Código

remover(_,[],[]).

remover(Elemento,[Elemento|Tail],Lista_modificada):-

remover(Elemento,Tail,Lista_modificada).

remover(Elemento,[Head|Tail1],[Head|Tail2]):-

Elemento\

=Head,remover(Elemento,Tail1,Tail2).

2.10.3 Descrição

De início, define-se como fato a remoção de um elemento qualquer, '_', de uma lista vazia, '[]', como a própria lista vazia. Afinal, não existem elementos na lista para serem removidos.

Seguindo adiante, inicia-se a regra 'remover', de aridade 3, na qual 'Elemento' corresponde ao item que se busca remover de uma lista, '[Elemento|Tail]' à lista original fornecida (na qual o primeiro elemento da lista é o mesmo elemento que se busca remover, 'Elemento', além dos demais termos da lista, 'Tail') e 'Lista_modificada' à lista em

que 'Elemento' foi removido. Depois disso, executa-se uma recursão enviando como parâmetros 'Elemento', 'Tail' e 'Lista_modificada'. Dessa forma, ao enviar-se a lista sem o primeiro elemento, este é removido da listagem.

Novamente, inicia a regra 'remove', de aridade 3, na qual 'Elemento' corresponde ao item que se busca remover (como na inicialização anterior), porém, a lista original '[Head|Tail]' agora possui 'Head' como subdivisão que indica seu primeiro elemento (não mais correspondendo ao elemento que se busca remover), além da lista modificada, '[Head|Tail_modificado]', ter sido dividida entre seu primeiro elemento, 'Head' (comum também à lista original), e os demais elementos, 'Tail_modificado'. Então, é verificado se 'Elemento' é diferente do primeiro elemento das listas, 'Head'. Por fim, realiza uma chamada recursiva, enviando ambas as listas sem seu elemento inicial, 'Head'. Dessa forma, os elementos da lista serão percorridos por completo e, quando 'Head' corresponder à 'Elemento', ele será removido utilizando a regra definida anteriormente. A recursão persiste até o último elemento da lista.

3 CONCLUSÃO

Dentre todas as dificuldades enfrentadas por nós ao decorrer do trabalho, a recursão e a manipulação de listas foram as maiores pedras no caminho. Pensar recursivamente certamente foi muito desafiador, pois, embora já fosse familiar o conceito de recursão em programações no escopo das linguagens compiladas, quando se é posto defronte uma estrutura e modo de pensar diferentes em uma linguagem declarativa, demora-se para acostumar com essa técnica. Porém, após praticar e ter a sintaxe do Prolog razoavelmente compreendida, pode-se prosseguir sem grandes empecilhos pelos exercícios que envolviam manipulação de listas. Mesmo detendo contato prévio como conceito de recursividade, muitos comandos e opções de manipular os dados comuns já não eram válidos para trabalhar com listas. Então, justamente a recursividade mostrou-se como uma segunda pedra no caminho, mas, lendo e consultado a documentação e o livro de assinalado nas referências foi possível finalizar as questões.

4 REFERÊNCIAS

LAGO, Silvio. **Introdução à Linguagem Prolog**. São Paulo, SP: IME-USP, 2003